
Table of Contents

PREAMBLE	1
=====	=====
1. COMMENTING	1
=====	=====
=====	=====
=====	3
2. FREQUENCY RESPONSE VS. CONVOLUTION	3
=====	=====
=====	3
=====	10
3. AUDIO FILTERING	10
=====	=====
=====	10
=====	16
4. AI FOR FILTERING	16
=====	=====
=====	16
ALL FUNCTIONS SUPPORTING THIS CODE %%	18

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

```
=====
=====
```

1. COMMENTING

```
=====
=====
```

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab05_comment.m')
```

```
%% USER-DEFINED VARIABLES
```

```
w = -pi:(pi/100):pi;
% <-- Answer: Why is w from -pi to pi?
% The discrete-time frequency variable w is periodic with period 2pi,
% so the interval from -pi to pi represents one complete fundamental cycle.
% Plotting over this range shows the full frequency response without
redundancy.
% negative pi to pi is also the principal aliasing interval
```

```

%

%% HIGHPASS FILTER

% FREQUENCY RESPONSE
H2 = (1-exp(-1j*w*1));
% <-- Answer: What is the difference equation for this frequency response?

% -----> INSERT ANSWER HERE <-----
%  $H(e^{j\omega}) = 1 - e^{-j\omega}$  corresponds to  $h[n] = \delta[n] - \delta[n-1]$ 
% Therefore, the difference equation is:
%  $y[n] = x[n] - x[n-1]$ 

% PLOT
figure;
subplot(2,1,1)
plot(w,abs(H2)); % ==> What does the abs() function do? <== abs() returns
the absolute value/magnitude of a value. (|H(e^{j\omega})|)
grid on;
title('Magnitude Response')
xlabel('Normalized Radian Frequency');
ylabel('Amplitude');
subplot(2,1,2)
plot(w,angle(H2)); % ==> What does the angle() function do? <== angle()
returns the phase angle of a complex number.
grid on;
title('Phase Response')
xlabel('Normalized Radian Frequency');
ylabel('Phase');

% <-- Answer: If you input a DC value into a highpass filter, what will be
%             its amplitude?

% A DC signal has frequency  $\omega = 0$ .
% A high-pass filter rejects low frequencies including DC.
% Therefore, the output amplitude will be zero.
%  $H(e^{j0}) = 1 - e^{-j \cdot 0} = 1 - 1 = 0$ 

```

2. FREQUENCY RESPONSE VS. CONVOLUTION

```
% -----
% 2 (a)
% -----
% Only need to modify function -- this area can be empty
% Create the function H = FreqResponse(b,nk,w) that outputs an FIR filter's
frequency response:
% where the input b is a vector of N filter coefficients, input w is a
vector of angular frequencies,
% nk is the k-th time sample (i.e., the k-th value in nk time vector), and
output H is the
% complex-valued frequency response. Include this function at the end of the
skeleton file.

% -----
% 2 (b)
% -----
% Plot the magnitude and phase responses for filter coefficients (i.e., an
impulse response)
% defined by b = [-1,-2,-1,1,2,1,-1,-2,-1] with corresponding time indices
nk = [0,1,2,3,4,5,6,7,8].
% Use your FreqResponse function.
b = [-1,-2,-1,1,2,1,-1,-2,-1];
nk = [0,1,2,3,4,5,6,7,8];

w = -pi:(pi/100):pi;

H = FreqResponse(b, nk, w);

figure;
%plot magnitude
subplot(2,1,1)
plot(w, abs(H));
grid on;
title('Magnitude Response')
xlabel('Normalized Radian Frequency');
ylabel('Amplitude');

%plot phase
```

```

subplot(2,1,2)
plot(w, angle(H));
grid on;
title('Phase Response')
xlabel('Normalized Radian Frequency');
ylabel('Phase');

% -----
% 2(c)
% -----
% Plot the magnitude and phase responses for filter coefficients (i.e., an
% impulse response)
% defined by b = [-1,-2,-1,1,2,1,-1,-2,-1] with corresponding time indices
nk = [-4,
% 3,-2,-1,0,1,2,3,4]. Use your FreqResponse function.

b = [-1,-2,-1,1,2,1,-1,-2,-1];
nk = [-4,-3,-2,-1,0,1,2,3,4];

w = -pi:(pi/100):pi;

H = FreqResponse(b, nk, w);

figure;

%plot magnitude
subplot(2,1,1)
plot(w, abs(H));
grid on;
title('Magnitude Response')
xlabel('Normalized Radian Frequency');
ylabel('Amplitude');

%plot phase
subplot(2,1,2)
plot(w, angle(H));
grid on;
title('Phase Response')
xlabel('Normalized Radian Frequency');
ylabel('Phase');

% -----
% 2(d)
% -----
% ANSWER QUESTION BELOW
% Answer in your comments: How do the two previous frequency responses
% differ? Why?
%
% The magnitude responses are identical, while the phase responses differ.
% This occurs because the filter coefficients are the same, but the time
% indices are shifted. A time shift does not affect magnitude, but it
% introduces a linear phase shift in the frequency response.

```

```

% -----
% 2(e)
% -----
% Plot the first four fundamental periods of the input signal x[n]
%  $x[n] = 1 + 2\cos((\pi/3)n) - \cos((2\pi/3)n + \pi/4)$  ,
% Use a stem plot. Label the horizontal axis "Samples" and the vertical axis
% "x[n]". Use the
% stem function. Store these first four fundamental periods into a variable x
n = 0:23; %N = 6 so 4N = 24 = 0:23

x = 1 + 2*cos((pi/3)*n) - cos((2*pi/3)*n + pi/4);

figure;
stem(n, x);
grid on;
xlabel('Samples');
ylabel('x[n]');
title('Input Signal x[n]');

% -----
% 2(f)
% -----
% Display (using the disp function) the frequency response for each
% frequency found in the
% input signal x[n]. Assume nk = [0,1,2,3,4,5,6,7,8]. Use your FreqResponse
% function.
% Make it clear what frequency corresponds to each frequency response value.

b = [-1,-2,-1,1,2,1,-1,-2,-1];
nk = [0,1,2,3,4,5,6,7,8];

% Frequencies present in x[n]
w0 = 0;           % DC component
w1 = pi/3;        % First cosine
w2 = 2*pi/3;      % Second cosine

% Evaluate frequency response at these frequencies
H0 = FreqResponse(b, nk, w0);
H1 = FreqResponse(b, nk, w1);
H2 = FreqResponse(b, nk, w2);

disp(['H(e^{j0})           = ', num2str(H0)])
disp(['H(e^{j(pi/3)})      = ', num2str(H1)])
disp(['H(e^{j(2pi/3)})     = ', num2str(H2)])
% -----
% 2(g)
% -----
% Plot the first four fundamental periods of the system output y[n] for an
% input of x[n] based
% on your values from FreqResponse function (i.e., do not use conv). Use the
% stem function

```

```

n = 0:23;

% frequency response values from 2(f)
% H0 = -4;
% H1 = -4.5 + 7.7942i;      % at w = pi/3
% H2 = 0.5 + 0.86603i;      % at w = 2pi/3

H0 = FreqResponse(b, nk, w0);
H1 = FreqResponse(b, nk, w1);
H2 = FreqResponse(b, nk, w2);

% H0
% H1
% H2

% Build y[n] using complex exponentials, then take real part
y = H0 ...
    + 2*real( H1 .* exp(1j*(pi/3)*n) ) ...
    - real( H2 .* exp(1j*((2*pi/3)*n + pi/4)) );

figure;
stem(n,y); grid on;
xlabel('Samples'); ylabel('y[n]');
title('Output Signal y[n] (Frequency Response)');

disp(['Mean of y (freq response method): ', num2str(mean(y))])

% -----
% 2(h)
% -----
% Plot (for the same time range as the previous subquestions) the system
output y[n] by con
% volving input stored in x with the filter impulse response b. Use the stem
function
b = [-1,-2,-1,1,2,1,-1,-2,-1];

y_conv_full = conv(x, b);      % full linear convolution (length = length(x)
+length(b)-1)

n = 0:23;                      % same time range as 2(e) and 2(g)
y_conv = y_conv_full(1:24);    % take first 24 samples to match n = 0:23

figure;
stem(n, y_conv);
grid on;
xlabel('Samples');
ylabel('y[n]');
title('Output Signal y[n] (Convolution)');

disp(['Mean of y_conv (convolution method): ', num2str(mean(y_conv))])

% -----
% 2(i)

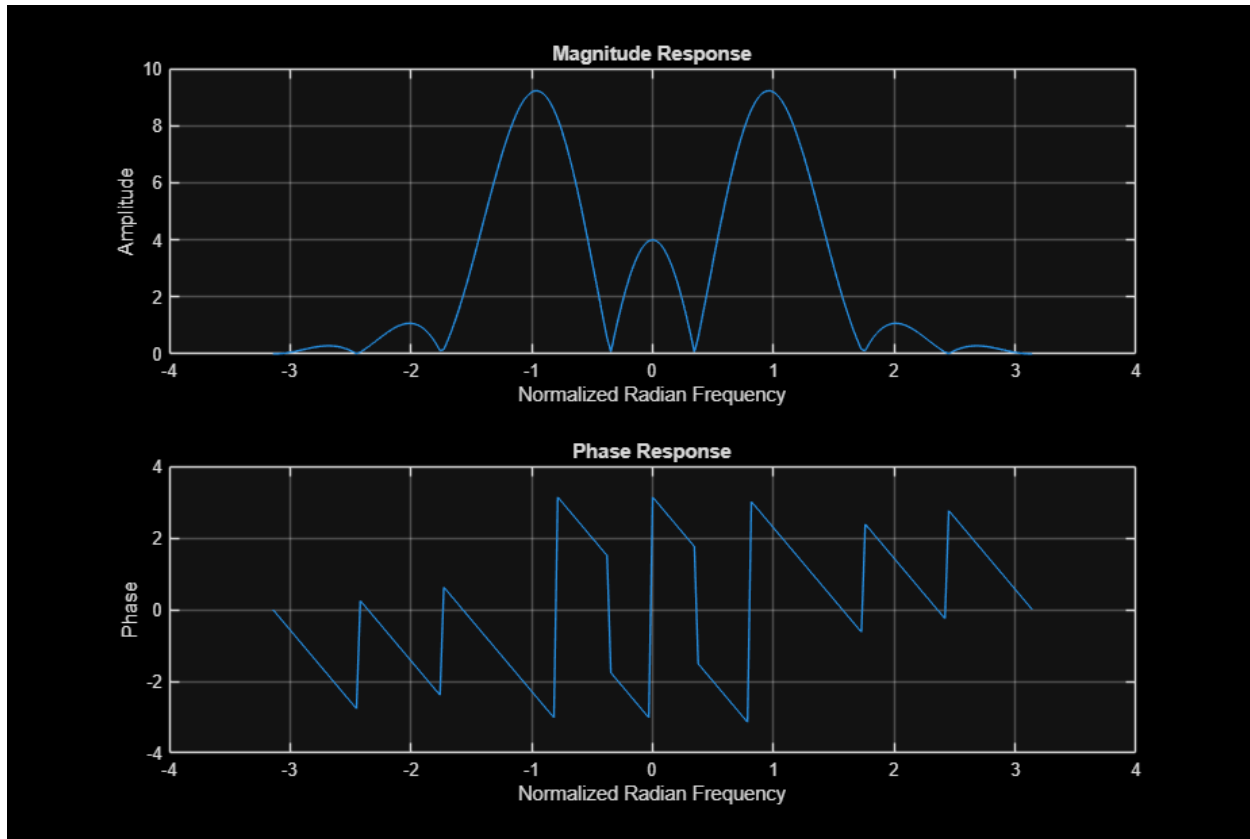
```

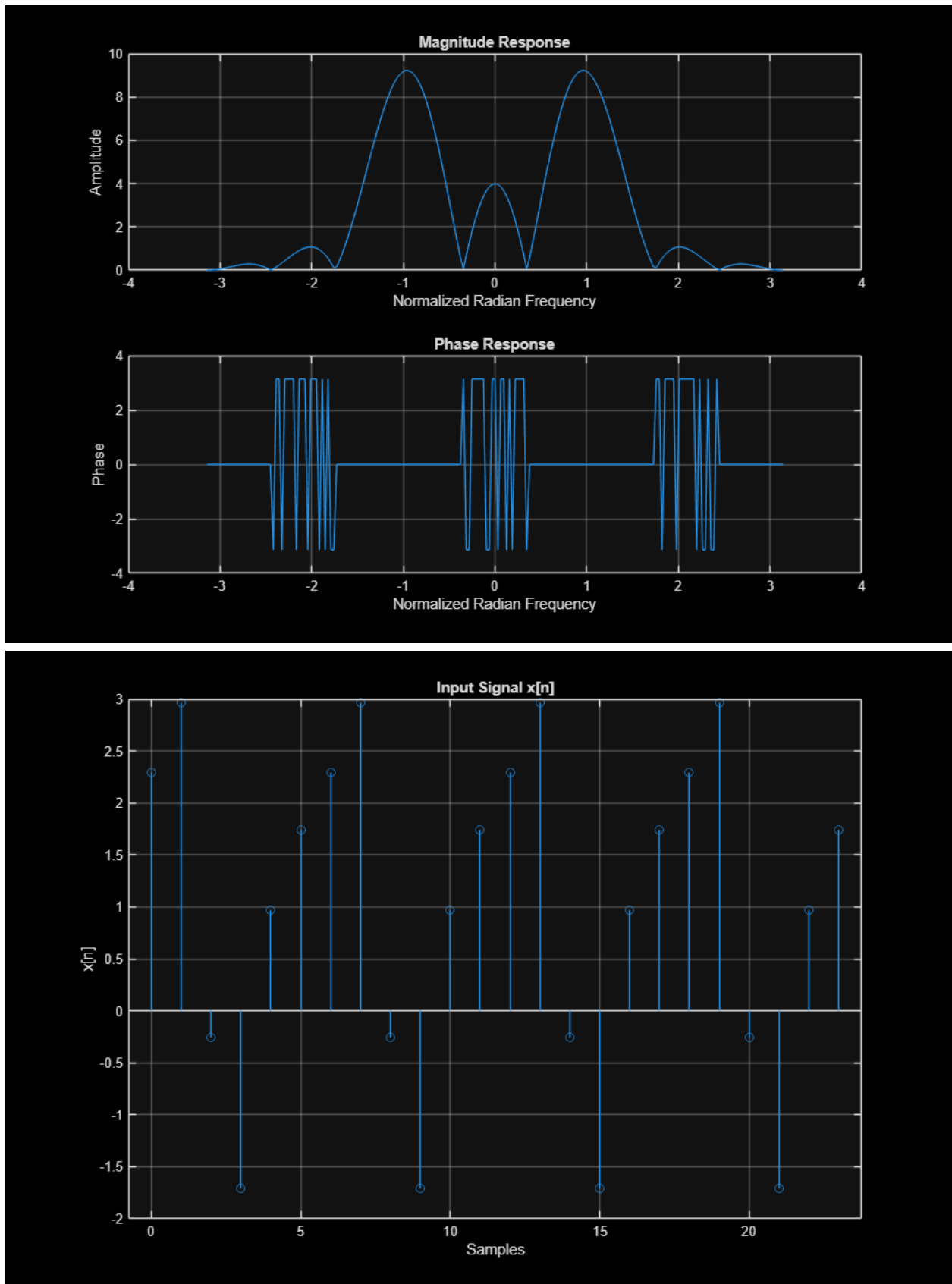
```

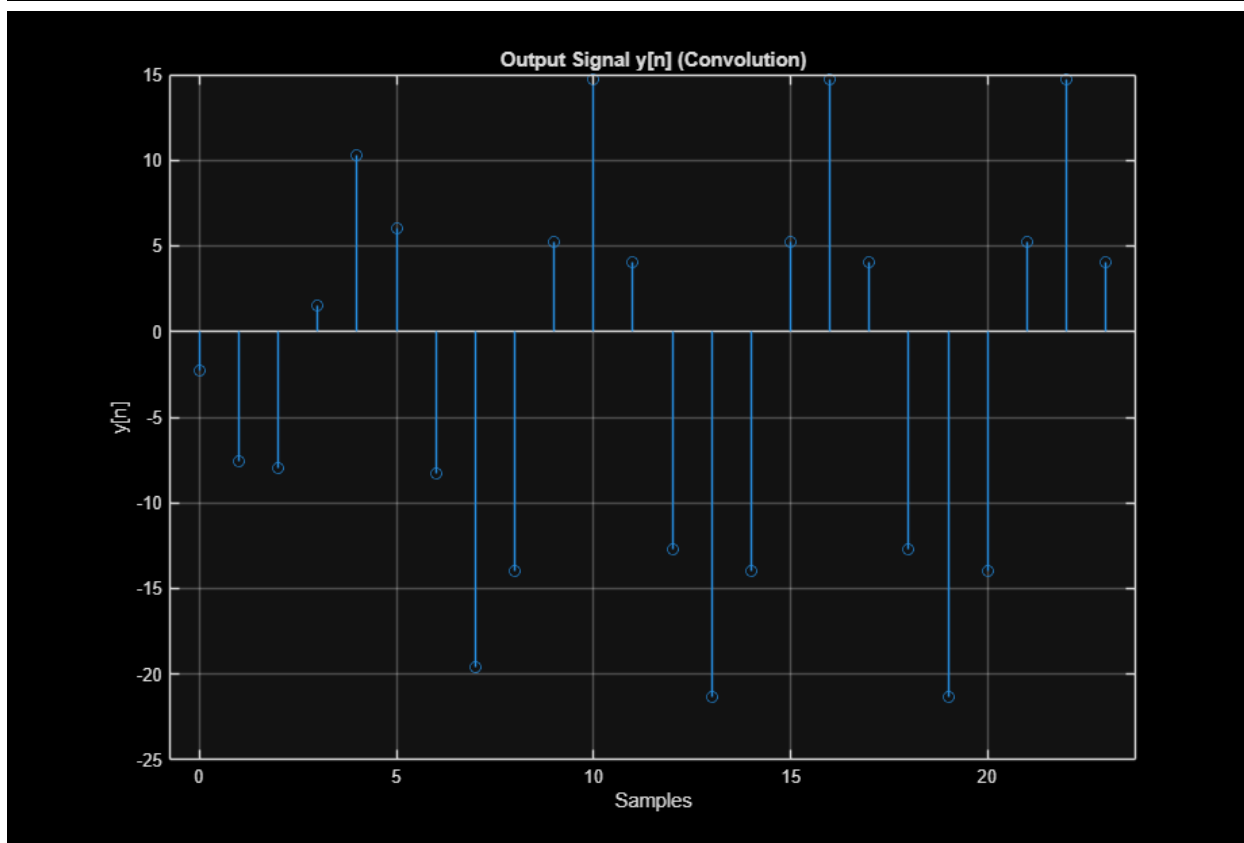
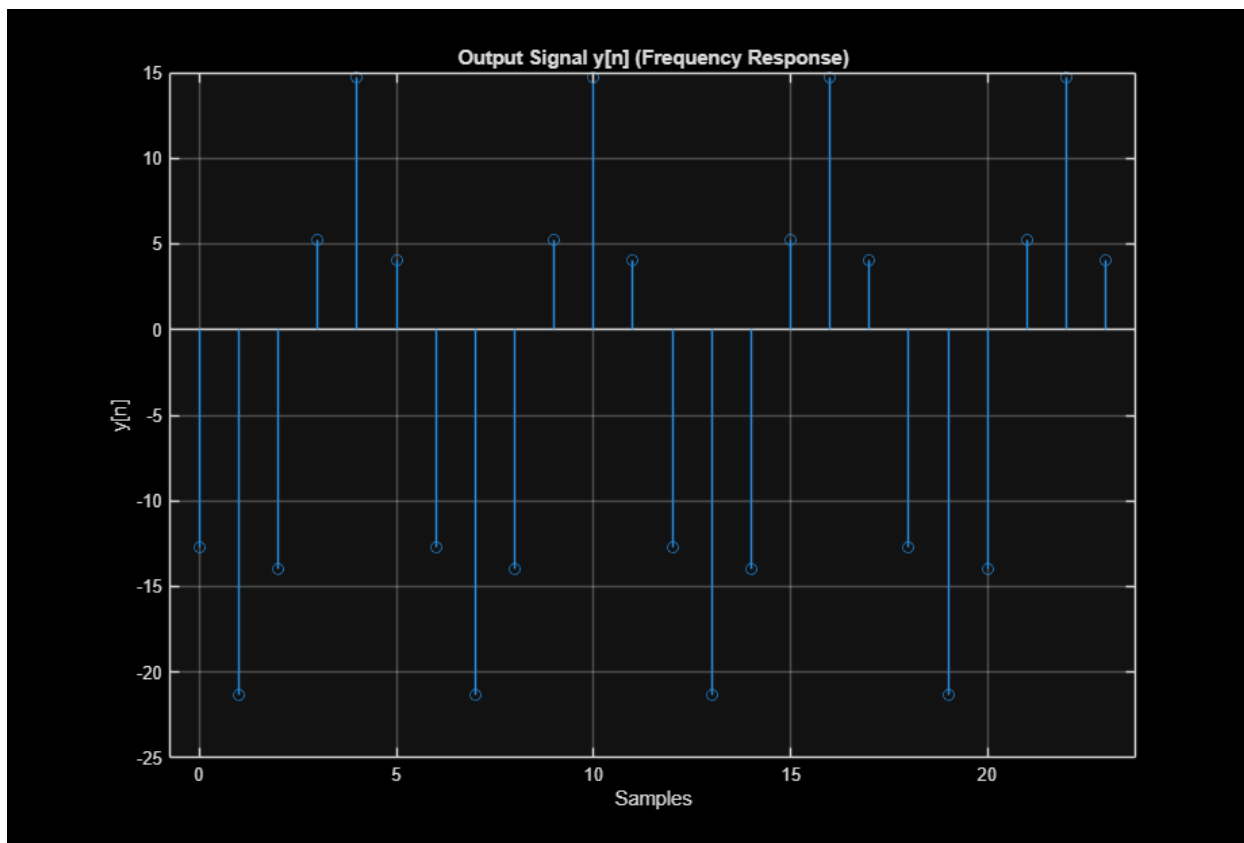
% -----
% ANSWER QUESTION BELOW
% Answer in your comments: How do the last two plots differ and why?

% % The plots are similar but not identical. The frequency-response method
% assumes an infinite periodic signal and produces the steady-state output,
% while convolution was performed on a finite-length signal. Truncation
% and zero-padding introduce boundary (edge) effects, causing small
% differences, particularly near the beginning and end of the signal.
%

```







3. AUDIO FILTERING

```
% MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE
% Audio from: https://freemusicarchive.org/music/lite-saturation/single/
logo/
% Title: Logo
% Artist: Lite Saturation
[x, fs] = audioread('lite_saturation.wav');

% FILTER a
a = [ ...
    0.0200    0.0191    0.0163    0.0120    0.0064         0   -0.0066
   -0.0130 ...
   -0.0185   -0.0225   -0.0247   -0.0247   -0.0222   -0.0172
   -0.0097         0 ...
    0.0115    0.0244    0.0380    0.0517    0.0647    0.0765    0.0863
    0.0938 ...
    0.0984    0.1000    0.0984    0.0938    0.0863    0.0765    0.0647
    0.0517 ...
    0.0380    0.0244    0.0115         0   -0.0097   -0.0172   -0.0222
   -0.0247 ...
   -0.0247   -0.0225   -0.0185   -0.0130   -0.0066         0    0.0064
    0.0120 ...
    0.0163    0.0191     0
  ];

% FILTER b
b = [ ...
    0.0200    0.0181    0.0128    0.0051   -0.0039   -0.0124   -0.0188
   -0.0221 ...
   -0.0212   -0.0162   -0.0076    0.0033    0.0147    0.0247    0.0311
    0.0324 ...
    0.0272    0.0153   -0.0029   -0.0262   -0.0524   -0.0790   -0.1034
   -0.1230 ...
   -0.1356    0.8600   -0.1356   -0.1230   -0.1034   -0.0790   -0.0524
   -0.0262 ...
   -0.0029    0.0153    0.0272    0.0324    0.0311    0.0247    0.0147
    0.0033 ...
   -0.0076   -0.0162   -0.0212   -0.0221   -0.0188   -0.0124   -0.0039
    0.0051 ...
    0.0128    0.0181     0
  ];
```

```

% -----
% 3(a)
% -----
nk = -25:25;           % index range (given)
w  = -pi:(pi/100):pi; % frequency axis

Ha = FreqResponse(a, nk, w); % compute frequency response

figure;
subplot(2,1,1)
plot(w, abs(Ha));
grid on;
title('Magnitude Response of a[n]');
xlabel('Normalized Radian Frequency');
ylabel('|H_a(e^{j\omega})|');

subplot(2,1,2)
plot(w, angle(Ha));
grid on;
title('Phase Response of a[n]');
xlabel('Normalized Radian Frequency');
ylabel('Phase');
% -----
% 3(b)
% -----
% ANSWER QUESTION BELOW
% Answer in your comments: Is this a lowpass, highpass, bandpass, or
bandstop filter?
%
% This filter is a lowpass filter. The magnitude response is largest
% near w = 0 and decreases at higher frequencies, indicating that
% low-frequency components are passed while high-frequency components
% are reduced.
%

% -----
% 3(c)
% -----
% Save (as xa.wav) the convolution of the input (in the skeleton) with the
filter coefficients a

xa = conv(x, a);           % convolve audio with filter a

xa = xa ./ max(abs(xa));   % normalize to prevent clipping

audiowrite('xa.wav', xa, fs); % save filtered audio
% -----
% 3(d)
% -----
% ANSWER QUESTION BELOW

```

```

% How does x (original sound) and xa (filtered sound) differ?
% Relate this difference to the filter magnitude response.
%
% % The filtered signal xa sounds more muffled than the original x.
% High-pitched content is greatly reduced, while lower-frequency
% components (bass) are emphasized. This matches the magnitude
% response of a[n], which is large near  $w = 0$  and small at higher
% frequencies, so the filter passes low frequencies and attenuates highs.
% The second one reduced a lot of the high-pitched noises and emphasized
% what sounded like a deep grumble.

% -----
% 3(e)
% -----
% Plot the magnitude and phase responses for the filter with coefficients /
% impulse response b
% (or b[n]) in the skeleton code. Use your FreqResponse function.

nk = -25:25;
w = -pi:(pi/100):pi;

Hb = FreqResponse(b, nk, w); % frequency response of b[n]

figure;
subplot(2,1,1)
plot(w, abs(Hb));
grid on;
title('Magnitude Response of b[n]');
xlabel('Normalized Radian Frequency');
ylabel('|H_b(e^{j\omega})|');

subplot(2,1,2)
plot(w, angle(Hb));
grid on;
title('Phase Response of b[n]');
xlabel('Normalized Radian Frequency');
ylabel('Phase');

% -----
% 3(f)
% -----
% ANSWER QUESTION BELOW
% Is this a lowpass, highpass, bandpass, or bandstop filter?
%
% This filter is a highpass filter. The magnitude response is small
% near  $w = 0$  and increases at higher frequencies, indicating that
% low-frequency components are attenuated while high-frequency
% components are passed.

%also included audio comparison to backup my answer:
% xb (filtered by b[n]) sounds brighter/sharper with more high-pitched
% content,

```

```

% and the low-frequency/bass components are reduced. This matches the
magnitude
% response of b[n], which is small near w = 0 and larger at higher
frequencies,
% so low frequencies are reduced while high frequencies are passed.

% -----
% 3(g)
% -----
% Save (as xb.wav) the convolution of the input (in the skeleton) with the
filter coefficients b.
xb = conv(x, b);           % convolve input with filter b

xb = xb ./ max(abs(xb));   % normalize to prevent clipping

audiowrite('xb.wav', xb, fs); % save filtered audio

% -----
% 3(h)
% -----
% Plot the magnitude and phase responses for the filter with coefficients /
impulse response
%  $c[n] = \delta[n] - b[n] - a[n]$ 

nk = -25:25;
w  = -pi:(pi/100):pi;

d = zeros(1,51);          % delta[n] over -25:25
d(nk==0) = 1;             %  $\delta[0] = 1$  at  $n = 0$ 

c = d - b - a;            %  $c[n] = \delta[n] - b[n] - a[n]$ 
Hc = FreqResponse(c, nk, w);

figure;
subplot(2,1,1)
plot(w, abs(Hc)); grid on;
title('Magnitude Response of c[n]');
xlabel('Normalized Radian Frequency');
ylabel('|H_c(e^{j\omega})|');

subplot(2,1,2)
plot(w, angle(Hc)); grid on;
title('Phase Response of c[n]');
xlabel('Normalized Radian Frequency');
ylabel('Phase');

% -----
% 3(i)
% -----
% ANSWER QUESTION BELOW
% Is this a lowpass, highpass, bandpass, or bandstop filter?

```

```

% Explain why we get this result.
%
% This is a bandpass filter. Its magnitude response is small near  $\omega = 0$ 
% (DC/low frequencies rejected) and also small near  $|\omega| \approx \pi$  (high
frequencies
% rejected), but it is large over a middle range of frequencies.
% This happens because  $H_c(e^{j\omega}) = 1 - H_b(e^{j\omega}) - H_a(e^{j\omega})$ ; since  $a[n]$ 
% is lowpass and  $b[n]$  is highpass, subtracting them from 1 leaves mainly the
% mid-frequency content.

```

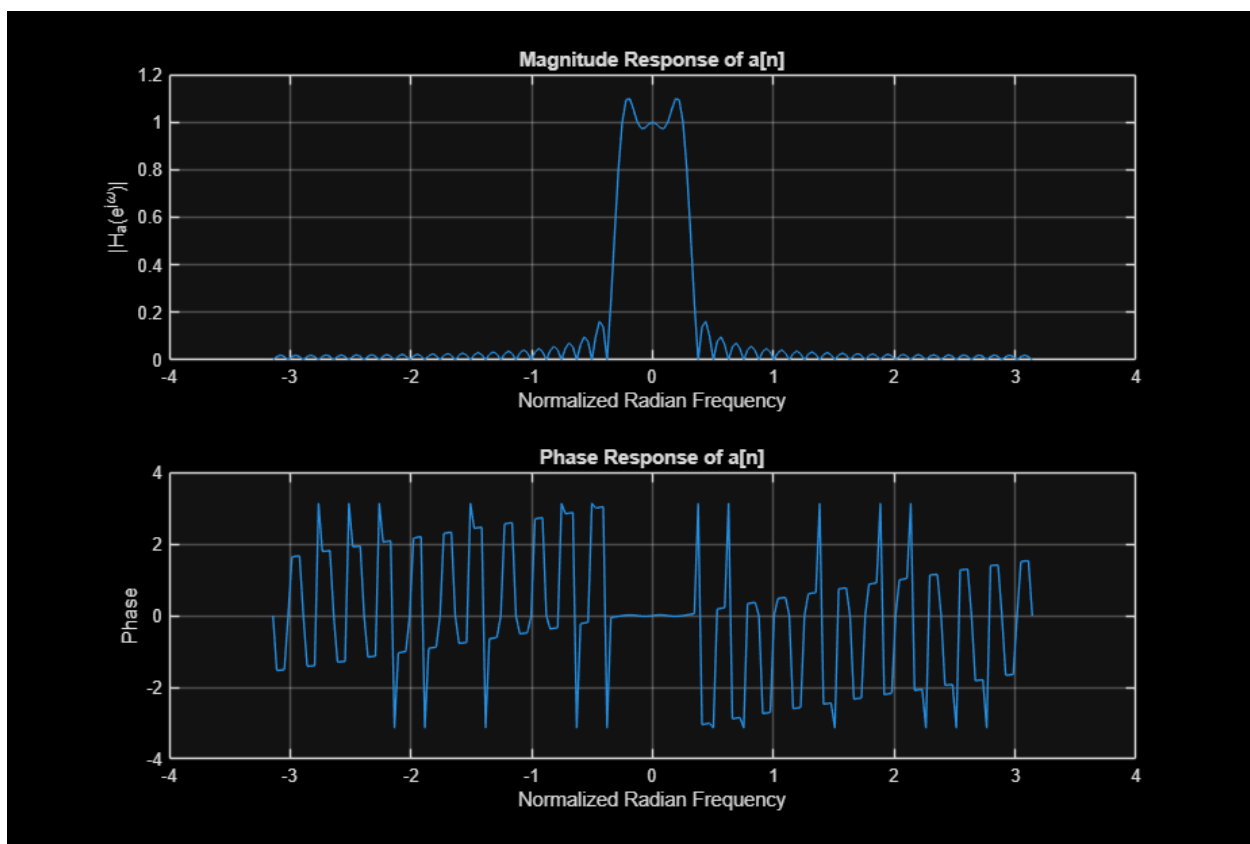
```

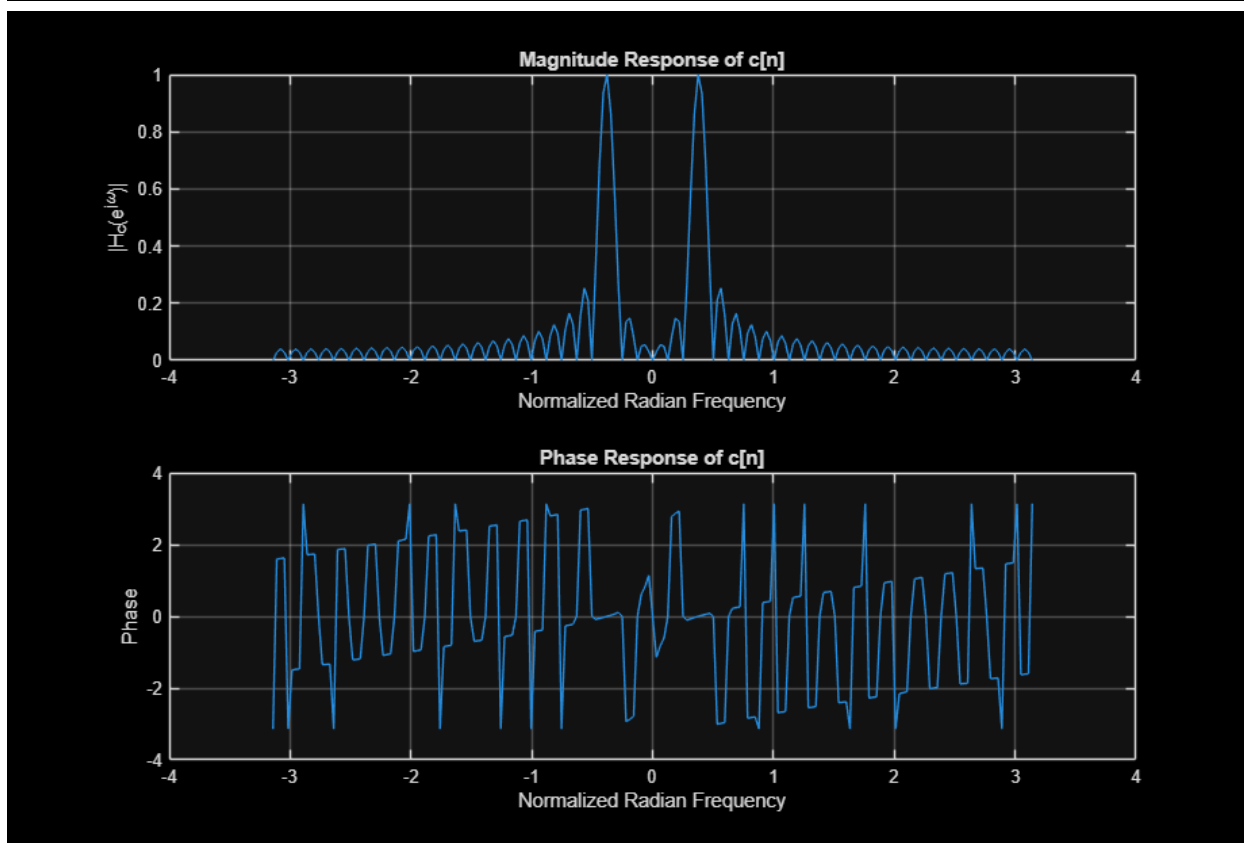
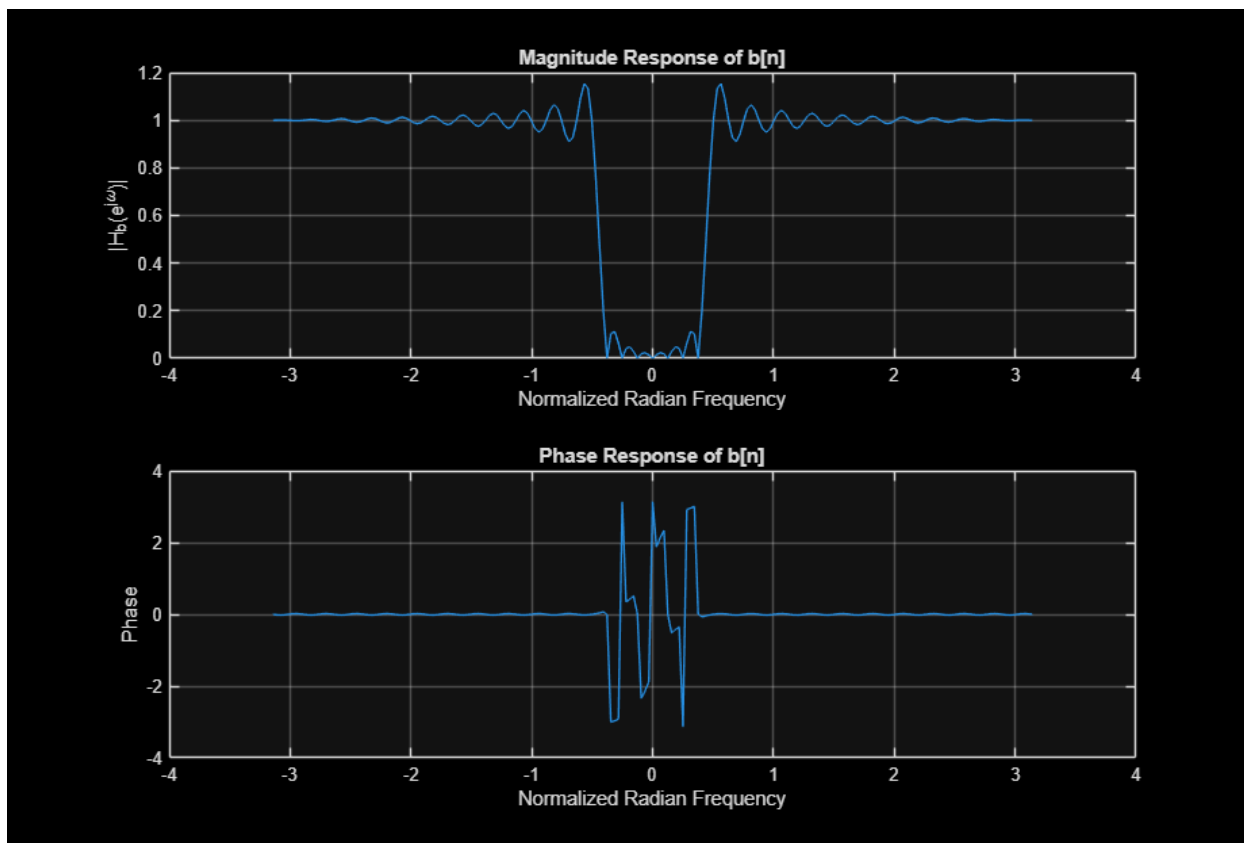
% -----
% 3(j)
% -----
% Save (as xc.wav) the convolution of the input (in the skeleton) with the
filter coefficients c
% (from c[n])
%
xc = conv(x, c); % convolve input with filter c

xc = xc ./ max(abs(xc)); % normalize to prevent clipping

audiowrite('xc.wav', xc, fs); % save filtered audio

```





4. AI FOR FILTERING

----- 4(a) ----- Create a function `y = highpass_ai(x,fs)` that inputs audio signal `x` and sampling rate `fs` and outputs a signal where frequencies below 500 Hz are strongly reduced. Below are the specific requirements • The function must use a digital filter implemented only with MATLAB's `conv` function. • The filter must be FIR. • The filter should strongly reduce low frequencies below 500 Hz. • The filter should mostly keep higher frequencies unchanged. • The filter should use 50 filter coefficients Based on your experience in the previous labs, design the prompt yourself.

```
% -----  
% 4 (b)  
% -----  
% Include your AI prompt and generated code as comments in  
% your MATLAB file.  
  
% Create a MATLAB function y = highpass_ai(x, fs) that designs and applies  
% an FIR high-pass filter using ONLY MATLAB's conv function.  
%  
% Requirements:  
% • The filter must be FIR  
% • Use exactly 50 filter coefficients  
% • Strongly reduce low frequencies below 500 Hz  
% • Preserve higher frequencies as much as possible  
% • The function should compute the impulse response internally (do not  
% hardcode coefficients)  
% • Use a windowed-sinc design method  
% • Must work for mono and stereo inputs  
% • Normalize output to avoid clipping  
%  
% Return only the MATLAB function code.  
  
% function [y,h] = highpass_ai(x,fs)  
% % HIGHPASS_AI 50-tap FIR high-pass filter (~500 Hz cutoff) using only  
% conv.  
% % Returns:  
% %   y = filtered audio (same length as x)  
% %   h = FIR coefficients (length 50)  
%  
% %% 1) Design the FIR high-pass filter (windowed-sinc)  
% N = 50;           % number of taps  
% fc = 500;         % cutoff (Hz)
```

```

% wc = 2*pi*fc/fs; % cutoff rad/sample
%
% n = 0:N-1;
% t = n - (N-1)/2; % centered indices (fractional center since N is even)
%
% % Ideal low-pass impulse response (no toolbox)
% h_lp = zeros(1,N);
% nz = (t ~= 0);
% h_lp(nz) = sin(wc*t(nz)) ./ (pi*t(nz));
% h_lp(~nz) = wc/pi; % limit as t -> 0
%
% % High-pass via spectral inversion: h_hp = delta - h_lp
% h_delta = zeros(1,N);
% centerIdx = floor(N/2) + 1; % for N=50, this is 26
% h_delta(centerIdx) = 1;
%
% h_hp = h_delta - h_lp;
%
% % Manual Hamming window (no toolbox)
% w_win = 0.54 - 0.46*cos(2*pi*(0:N-1)/(N-1));
% h = h_hp .* w_win;
%
% %% 2) Filter the audio (conv only)
% wasVector = isvector(x);
% if wasVector
%     x = x(:); % column
% end
%
% y = zeros(size(x));
% for ch = 1:size(x,2)
%     y(:,ch) = conv(x(:,ch), h, 'same');
% end
%
% %% 3) Normalize to avoid clipping
% mx = max(abs(y(:)));
% if mx > 0
%     y = y / (mx + eps);
% end
%
% if wasVector
%     y = y(:); % keep as column vector (Matlab audio is usually Nx1)
% end
% end

% -----
% 4(c)
% -----
% Apply your AI-designed high-pass filter
[y_ai, h] = highpass_ai(x, fs);

% Save to WAV file

```

```

audiowrite('filtered_ai.wav', y_ai, fs);

disp('filtered_ai.wav saved successfully');

% -----
% 4(d)
% -----
% Plot the magnitude and phase responses for your filter. Use your
% FreqResponse function.
% Frequency axis

w = -pi:(pi/200):pi;
nk = 0:49;

H_ai = FreqResponse(h, nk, w);

figure;
subplot(2,1,1)
plot(w, abs(H_ai));
grid on;
title('Magnitude Response of AI High-Pass Filter');
xlabel('Normalized Radian Frequency');
ylabel('|H_{ai}(e^{j\omega})|');

subplot(2,1,2)
plot(w, angle(H_ai));
grid on;
title('Phase Response of AI High-Pass Filter');
xlabel('Normalized Radian Frequency');
ylabel('Phase');

snapnow;

filtered_ai.wav saved successfully

```

ALL FUNCTIONS SUPPORTING THIS CODE % %

```

function H = FreqResponse(b,nk,w)
% ==> Describe function here <==
% Computes the complex frequency response  $H(e^{jw})$  of an FIR filter
% b = filter coefficients, nk = time indices, w = frequencies

H = zeros(size(w)); % initialize response

for k = 1:length(b)
    H = H + b(k).*exp(-1j*w*nk(k));
end

end

```

```

function [y,h] = highpass_ai(x,fs)
% HIGHPASS_AI 50-tap FIR high-pass filter (~500 Hz cutoff) using only conv.
% Returns:
%   y = filtered audio (same length as x)
%   h = FIR coefficients (length 50)

% 1) Design the FIR high-pass filter (windowed-sinc)
N = 50;           % number of taps
fc = 500;         % cutoff (Hz)
wc = 2*pi*fc/fs;  % cutoff rad/sample

n = 0:N-1;
t = n - (N-1)/2;  % centered indices (fractional center since N is even)

% Ideal low-pass impulse response (no toolbox)
h_lp = zeros(1,N);
nz = (t ~= 0);
h_lp(nz) = sin(wc*t(nz)) ./ (pi*t(nz));
h_lp(~nz) = wc/pi; % limit as t -> 0

% High-pass via spectral inversion: h_hp = delta - h_lp
h_delta = zeros(1,N);
centerIdx = floor(N/2) + 1; % for N=50, this is 26
h_delta(centerIdx) = 1;

h_hp = h_delta - h_lp;

% Manual Hamming window (no toolbox)
w_win = 0.54 - 0.46*cos(2*pi*(0:N-1)/(N-1));
h = h_hp .* w_win;

% 2) Filter the audio (conv only)
wasVector = isvector(x);
if wasVector
    x = x(:); % column
end

y = zeros(size(x));
for ch = 1:size(x,2)
    y(:,ch) = conv(x(:,ch), h, 'same');
end

% 3) Normalize to avoid clipping
mx = max(abs(y(:)));
if mx > 0
    y = y / (mx + eps);
end

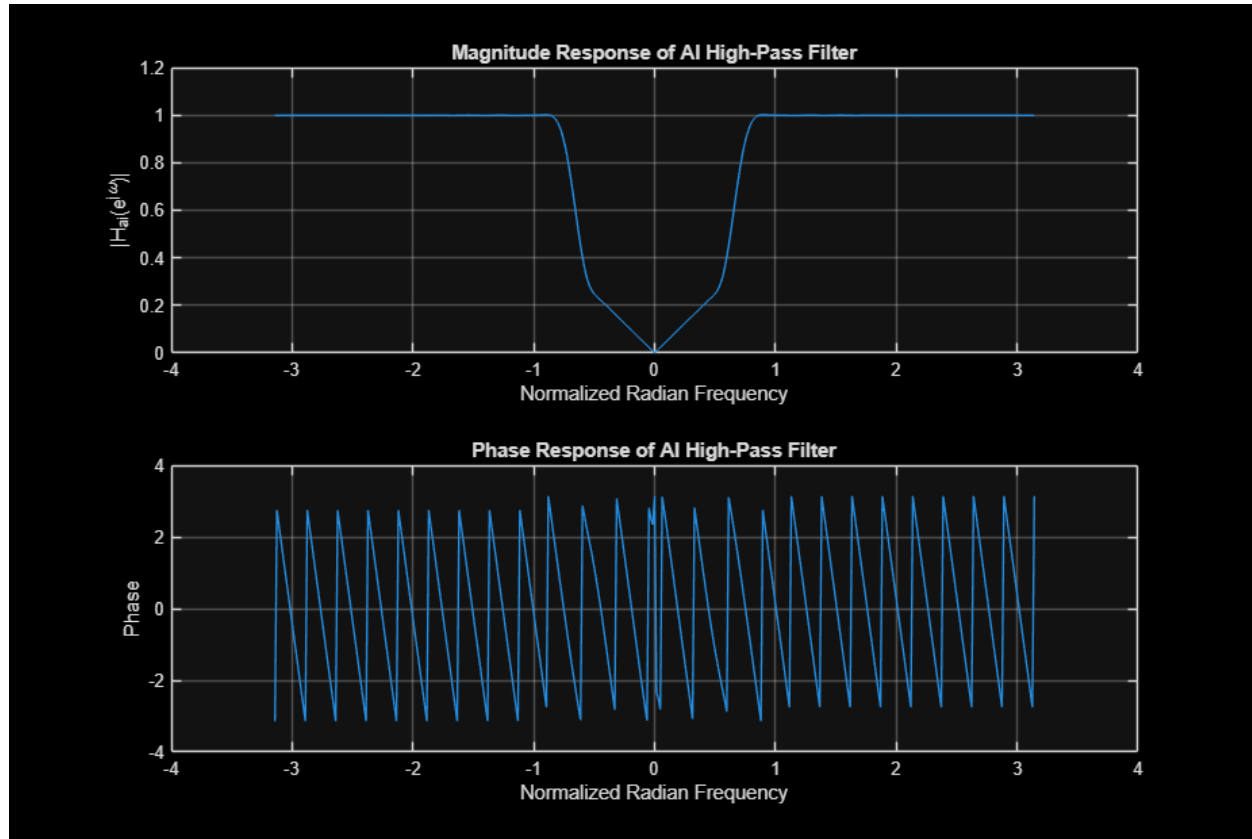
if wasVector
    y = y(:); % keep as column vector (Matlab audio is usually Nx1)
end
end

```

```

H(e^{j0})           = -4
H(e^{j(pi/3)})      = -4.5+7.7942i
H(e^{j(2pi/3)})     = 0.5+0.86603i
Mean of y (freq response method): -4
Mean of y_conv (convolution method): -2.7421

```



Published with MATLAB® R2025b